# LAPORAN RESMI PRAKTIKUM

## DevSecOps — Bab 4

### Web Service Container: Apache, Nginx, Reverse Proxy, dan TLS




Nama : Fawwaz Mufid Wardaya
Kelas : D4 Teknik Informatika B
NRP : 3126640050



**PROGRAM STUDI TEKNIK INFORMATIKA**
**DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER**
**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

**2026**

# Pendahuluan

## Latar Belakang

Web service adalah titik pertemuan antara client, jaringan, dan logika aplikasi. Dalam arsitektur container, Apache HTTP Server atau Nginx dapat berperan sebagai origin server, penyaji berkas statis, maupun reverse proxy. Reverse proxy memisahkan interface publik dari backend internal, memusatkan terminasi TLS, routing, penerapan header keamanan, dan pencatatan akses. Praktikum Bab 4 menjalankan Apache dan Flask di belakang Nginx, menerapkan sertifikat TLS self-signed untuk simulasi HTTPS, mengarahkan /api ke backend Flask, serta membaca access log dari bind mount.

## Rumusan Masalah

Mengapa reverse proxy tidak seharusnya menjalankan semua logic aplikasi?

Apa perbedaan TLS termination dan end-to-end TLS?

Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?

Apa konsekuensi menyimpan private key TLS di bind mount?

Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.

# Pembahasan

## Dasar Teori

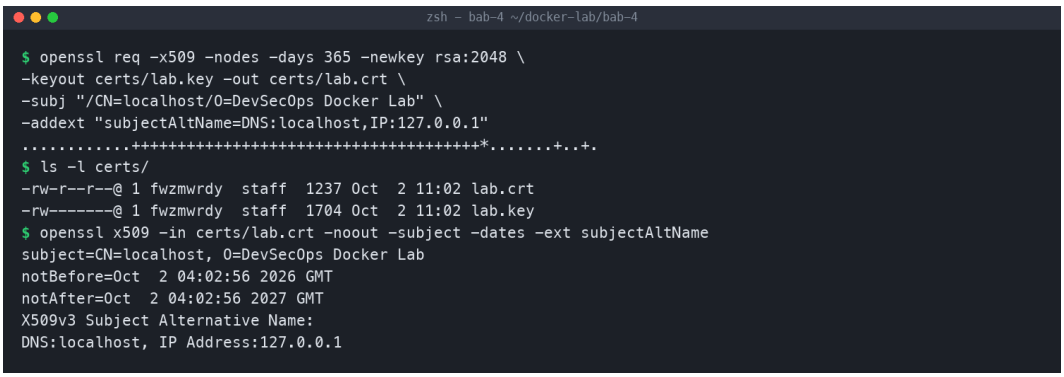
Reverse proxy menerima koneksi dari client, menerapkan kebijakan pada lapisan depan, lalu meneruskan permintaan ke service internal. Pola ini menyederhanakan terminasi TLS, routing berbasis host atau path, pembatasan ukuran request, pencatatan akses, dan penerapan header keamanan. Namun reverse proxy bukan pengganti kontrol aplikasi: validasi input, autentikasi, dan otorisasi tetap harus berada pada service yang memiliki konteks bisnis.

TLS melindungi kerahasiaan dan integritas data selama transit serta membantu client memverifikasi identitas server. Terminasi TLS pada proxy memusatkan pengelolaan sertifikat, tetapi menciptakan keputusan baru: apakah koneksi proxy ke backend tetap terenkripsi dan bagaimana private key disimpan. Sertifikat self-signed hanya cocok untuk laboratorium karena tidak dipercaya otomatis oleh client. Header terpercaya seperti X-Forwarded-Proto hanya boleh dipercaya bila request benar-benar datang dari proxy yang sah.

## Praktikum 1 — Sertifikat TLS self-signed dengan Subject Alternative Name

**Perintah:**

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/lab.key -out certs/lab.crt \
  -subj "/CN=localhost/O=DevSecOps Docker Lab" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
chmod 600 certs/lab.key
```



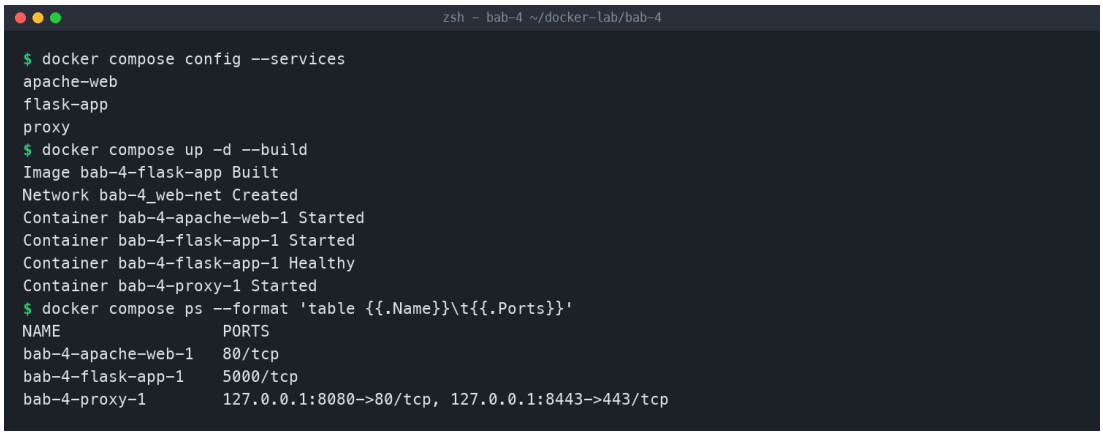
*Gambar 1. Sertifikat lab dibuat dengan SAN localhost dan permission private key 600.*

**Analisis:** Sertifikat memuat Common Name dan Subject Alternative Name DNS:localhost serta IP:127.0.0.1, sehingga handshake untuk kedua identitas tersebut dapat dicocokkan. Private key disimpan dengan permission 600 (hanya pemilik) sebagai langkah minimum. Sertifikat berlaku satu tahun, dan tanggal kedaluwarsa harus dipantau agar tidak menjadi insiden operasional.

## Praktikum 2 — Stack reverse proxy Nginx, Apache, dan Flask

**Perintah:**

```
docker compose config --services
docker compose up -d --build
docker compose ps
```




*Gambar 2. Hanya service proxy yang mempublikasikan port ke host.*

**Analisis:** Compose mengorkestrasi tiga service pada network web-net. Proxy menunggu flask-app berstatus healthy sebelum start berkat condition service_healthy. Hanya proxy yang mempublikasikan port, dan keduanya diikat ke loopback (127.0.0.1:8080 dan 127.0.0.1:8443), sedangkan apache-web (80/tcp) dan flask-app (5000/tcp) hanya memiliki port internal. Pola ini menerapkan least exposure: backend tidak dapat dijangkau langsung dari host.

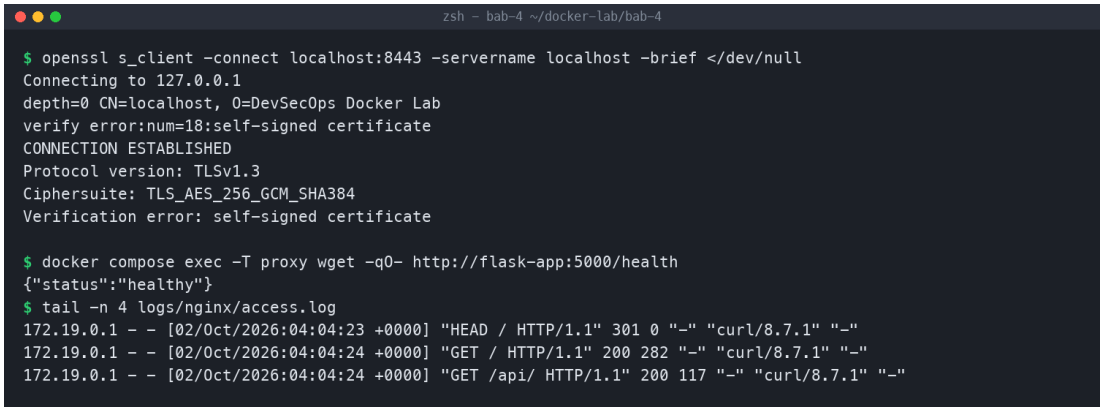
## Praktikum 3 — Redirect HTTP ke HTTPS, routing /api, dan verifikasi TLS

**Perintah:**

```
curl -I http://localhost:8080/
curl -k -s -i https://localhost:8443/
curl -k -s https://localhost:8443/api/
openssl s_client -connect localhost:8443 -servername localhost -brief </dev/null
```




*Gambar 3. HTTP dialihkan ke HTTPS, halaman Apache menyertakan header keamanan, /api menuju Flask.*

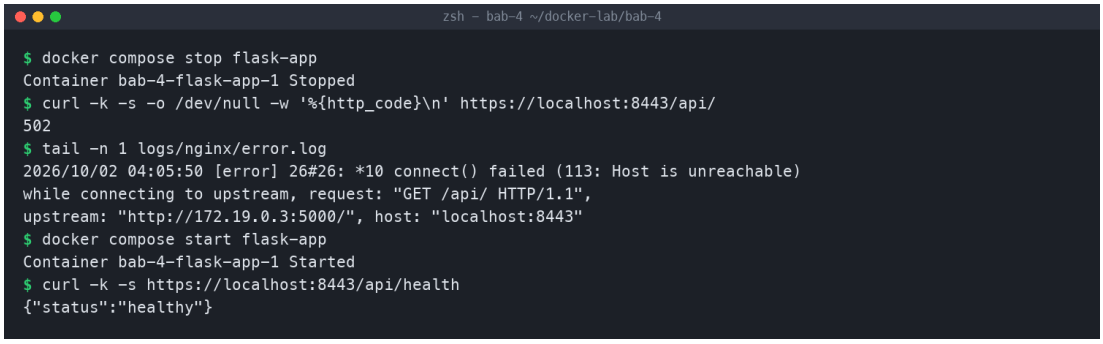



*Gambar 4. TLSv1.3 dinegosiasikan; isolasi upstream dan access log terbukti.*

**Analisis:** Permintaan ke port 8080 menghasilkan 301 ke https://localhost:8443/, memaksa kanal terenkripsi. Halaman Apache disajikan dengan header X-Content-Type-Options, X-Frame-Options, dan Referrer-Policy. Route /api/ diteruskan ke Flask dan mengembalikan forwarded_proto=https, membuktikan header X-Forwarded-Proto diteruskan dengan benar. Handshake openssl menegaskan negosiasi TLSv1.3 dengan cipher AES-256-GCM, sekaligus melaporkan Verification error self-signed certificate yang memang diharapkan pada lab. Upstream apache-web dan flask-app dapat diakses dari proxy melalui nama service, dan seluruh request tercatat pada access log di host.

## Analisis Masalah: 502 Bad Gateway Saat Backend Tidak Tersedia

Untuk menguji failure mode, service flask-app dihentikan. Permintaan ke https://localhost:8443/api/ kemudian mengembalikan 502 Bad Gateway. Diagnosis dilakukan berlapis: access log menunjukkan request diterima proxy dengan status 502, lalu error log menampilkan "connect() failed (113: Host is unreachable) while connecting to upstream ... http://172.19.0.3:5000/". Pesan ini membuktikan proxy meneruskan request ke upstream yang tidak dapat dijangkau, bukan kegagalan TLS atau Apache. Setelah flask-app dijalankan kembali dan healthcheck lulus, endpoint /api/health kembali mengembalikan healthy. Pelajaran: status 5xx dari reverse proxy harus ditelusuri ke lapisan upstream, dan readiness/retry perlu dirancang agar pemulihan tidak manual.




*Gambar 5. Backend mati menghasilkan 502; error log menunjuk upstream yang tidak terjangkau.*

## Troubleshooting

| Gejala | Penyebab yang mungkin | Tindakan korektif |
|---|---|---|
| 502 Bad Gateway | Upstream (flask-app/apache-web) mati atau belum siap | Cek logs proxy dan upstream; pulihkan service |
| HTTPS gagal / error sertifikat | Sertifikat self-signed tidak dipercaya | Gunakan -k di lab; di produksi pakai CA resmi |
| Redirect loop | X-Forwarded-Proto tidak diteruskan dengan benar | Pastikan header proxy_set_header diset |
| Log permission denied | Ownership logs/nginx tidak sesuai | Koreksi permission/ownership direktori log |
| Port 8080/8443 dipakai | Proses lain menempati port host | Ganti host port atau hentikan proses |

| | | |
|---|---|---|
| | | lama |

# Evaluasi dan Latihan Mandiri

**1. Mengapa reverse proxy tidak seharusnya menjalankan semua logic aplikasi?**

Reverse proxy berada pada edge dan tidak memiliki konteks bisnis seperti identitas pengguna, otorisasi, atau state aplikasi. Menempatkan logic di proxy menciptakan choke point monolitik, mempersulit pengujian, dan mencampur zona kepercayaan. Peran proxy adalah terminasi TLS, routing, pembatasan, dan header keamanan; validasi serta otorisasi tetap di aplikasi.

**2. Apa perbedaan TLS termination dan end-to-end TLS?**

TLS termination mengakhiri enkripsi di proxy sehingga lalu lintas proxy ke backend berjalan tanpa TLS; pengelolaan sertifikat terpusat dan backend lebih sederhana. End-to-end TLS mempertahankan enkripsi hingga backend, sehingga segmen internal juga terlindungi, tetapi setiap service harus mengelola sertifikat dan rotasinya.

**3. Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?**

Jangan definisikan ports pada apache-web dan flask-app sehingga hanya port internal (80/tcp, 5000/tcp) yang aktif, ikat port proxy ke loopback (127.0.0.1), dan tempatkan backend pada network privat bersama proxy. Sebagai penguatan, network dapat ditandai internal agar tidak memiliki jalur keluar. Bukti pada praktikum: hanya proxy yang memiliki pemetaan host.

**4. Apa konsekuensi menyimpan private key TLS di bind mount?**

Private key ikut tersimpan di filesystem host sehingga dapat dibaca proses lain yang memiliki akses, dan berpotensi bocor bila host dikompromikan atau mount bersifat longgar. Konsekuensinya penyerang dapat mengimpersonasi server. Mitigasi: mount read-only, permission minimum (600, pemilik root), gunakan Docker secrets atau secret manager, jangan commit key ke git, dan rotasi berkala.

**5. Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.**

Access log Nginx (combined) padat: satu baris per request berisi IP, waktu, method, path, status, ukuran, referer, dan user-agent, sehingga cepat dianalisis untuk traffic dan status. Pada praktikum, access log proxy di bind mount langsung menangkap request client beserta status 200/301/502. Apache memiliki format serupa (common/combined) plus error log dengan level, berguna untuk masalah modul dan .htaccess. Nginx unggul sebagai log lapisan proxy, sedangkan error log Apache lebih detail untuk origin server.

# Penutup

## Kesimpulan

Praktikum Bab 4 berhasil menempatkan Apache dan Flask di belakang Nginx sebagai reverse proxy. Sertifikat TLS self-signed dengan SAN localhost dan IP:127.0.0.1 memungkinkan simulasi HTTPS; handshake menegosiasikan TLSv1.3 dengan cipher AES-256-GCM. HTTP dialihkan ke HTTPS melalui 301, halaman Apache disajikan dengan header keamanan, dan /api/ diteruskan ke Flask dengan forwarded_proto=https. Isolasi terbukti karena hanya proxy yang mempublikasikan port pada loopback, sementara Apache dan Flask hanya aktif pada port internal. Access log tersimpan di host, dan kegagalan backend terdiagnosis cepat sebagai 502 melalui error log yang menunjuk upstream tidak terjangkau. Catatan keamanan utama: sertifikat lab tidak dipercaya client (harus memakai -k) dan

private key berada di bind mount, sehingga pada lingkungan production-like perlu CA terpercaya, Docker secrets, serta otomatisasi renewal.

# Daftar Pustaka

Materi Ajar DevOps (DevSecOps Terintegrasi) — Bab 4. Politeknik Elektronika Negeri Surabaya, 2026.

Docker Inc. (2026). Docker Documentation — Networking dan Compose. https://docs.docker.com/

Nginx Inc. (2026). Nginx Documentation — Reverse Proxy dan SSL/TLS. https://nginx.org/en/docs/

Akses terakhir: 2 Oktober 2026.